



INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	Estructuras de Selección
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	Primero - B
Alumnos participantes:	Ashanga Yumbo Brishy Anahy Calderón Carvajal Fher Dorian Casillas Ochoa Antony Sebastián Paredes Acosta Dereck Shair Sánchez Bastidas Karina Paola
Asignatura:	Algoritmos y Lógica de programación
Docente:	Ing. José Ruben Caizabuano, Mg.

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:

Utilizar estructuras de selección en el análisis y diseño de algoritmos.

Específicos:

- Identificar las estructuras de selección simples, dobles y múltiples utilizadas en el diseño de algoritmos.
- Aplicar estructuras de selección mediante condiciones lógicas para resolver problemas planteados en algoritmos.
- Diseñar algoritmos que utilicen estructuras de selección para tomar decisiones de manera correcta y eficiente.

2.2 Modalidad

Presencial

2.3 Tiempo de duración

Presenciales: 6

No presenciales: 0

2.4 Instrucciones

- El trabajo se desarrollará de manera individual.
- Lea detenidamente cada uno de los ejercicios planteados en el aula virtual y realice la codificación en C++.
- Subir en un documento en formato pdf que contenga capturas del código de cada ejercicio, adicionalmente, comprimir en una carpeta los archivos con el código generado, y subirlo al aula virtual.
- Si se manifiestan dudas se invita a los estudiantes a acudir a las tutorías académicas

2.5 Listado de equipos, materiales y recursos

Listado de equipos y materiales generales empleados en la guía práctica:

- TAC Inteligencia artificial
- TAC - Computador o laptop.
- Plataforma educativa de la UTA.
- Internet para consultas.



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO – DICIEMBRE 2026



- Papel
- Material de escritorio

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

- ☒ Plataformas educativas
- ☒ Simuladores y laboratorios virtuales
- ☒ Aplicaciones educativas
- ☒ Recursos audiovisuales
- ☐ Gamificación
- ☐ Inteligencia Artificial

Otros (Especifique): _____

2.6 Actividades por desarrollar

Desarrollar la guía APE en base al formato y las instrucciones brindadas

2.7 Resultados obtenidos

PRUEBAS DE ESCRITORIO

Resumen de los casos EJERCICIO 1 - CONTROL DE HORA

Caso	Entrada inicial	Corrección	Hora mostrada	Cambiar	Resultado
1	10, 30, 45	No aplica	10:30:45	0	Finaliza
2	23:59:59	00:00:00	Dos horas	1 → 0	Finaliza
3	25, -1, 60	14, 5, 9	14:05:09	0	Valida y finaliza

Resumen de los casos EJERCICIO 2 - SISTEMA DE CUENTAS BANCARIAS

Caso	Operación	Saldo inicial	Monto	Saldo final	Resultado
1	Recibir	\$100,00	\$50,00	\$150,00	Aceptado
2	Enviar	\$200,00	\$250,00 → \$50,00	\$150,00	Valida saldo
3	Transferir	\$300,00 / \$100,00	\$75,00	\$225,00 / \$175,00	Aceptado



Resumen de los casos EJERCICIO 3 - GESTIÓN DE UN JEAN EN FÁBRICA O TIENDA

Caso	Opción	Valor inicial	Operación	Valor final	Resultado
1	Mostrar	Datos completos	Sin cambios	Datos iguales	Muestra
2	Lavar	2 teñidos	2 - 1	1 teñido	Actualiza
3	Secar	5 %	5 - 10; mínimo 0	0 %	Jean seco

2.8 Habilidades blandas empleadas en la práctica

- ☒ Liderazgo
- ☒ Trabajo en equipo
- ☐ Comunicación asertiva
- ☐ La empatía
- ☒ Pensamiento crítico
- ☒ Flexibilidad
- ☒ La resolución de conflictos
- ☒ Adaptabilidad
- ☒ Responsabilidad

2.9 Conclusiones

- Se comprendió la importancia de las estructuras de selección en el diseño de algoritmos, ya que permiten que un programa tome decisiones de acuerdo con diferentes condiciones.
- Se aplicaron estructuras condicionales en situaciones prácticas, como el control de una hora, la gestión de cuentas bancarias y la administración de un jean, permitiendo resolver cada problema de manera lógica y organizada.
- Se fortaleció la capacidad para analizar y diseñar algoritmos, identificando las condiciones necesarias para ejecutar diferentes acciones y obtener resultados adecuados en los lenguajes C++ y Java.

2.10 Recomendaciones

- Analizar cuidadosamente el problema antes de programar, identificando las condiciones, datos de entrada y posibles resultados para seleccionar correctamente la estructura de selección que se utilizará.
- Practicar con diferentes ejercicios de estructuras condicionales, utilizando situaciones cotidianas que permitan comprender mejor el funcionamiento de if, else y otras estructuras de selección.
- Realizar pruebas con diferentes datos de entrada, incluyendo casos normales y casos límite, para comprobar que el algoritmo funcione correctamente y evitar errores durante la ejecución.



2.11 Referencias bibliográficas

- [1] F. J. C. Sierra, *Programación orientada a objetos con C++*. Madrid, España: Ra-Ma Editorial, 2018.
- [2] H. A. F. Fernández, *Programación orientada a objetos usando Java*. Bogotá, Colombia: Ecoe Ediciones, 2012.
- [3] J. O. R. D. I. Marco Gómez, A. N. G. E. L. A. Martín Prat, X. A. V. I. E. R. Molinero Albareda, P. P. Vázquez Alcocer, & F. Xhafa, *Programación en C++ para ingenieros*. Madrid, España: Ediciones Paraninfo, S. A., 2006.

2.12 Anexos

Link del GitHub: <https://github.com/Doros-ia/APE-02-Estructuras-de-Selección/tree/main>

CONTROL DE HORA

Análisis, algoritmo, pseudocódigo, flujograma, prueba de escritorio y codificación

ENUNCIADO DEL EJERCICIO

Desarrollar un programa en C++ y Java que permita ingresar una hora, minutos y segundos. Luego, el sistema debe mostrar la hora registrada y preguntar al usuario si desea cambiarla. Si responde que sí, se debe permitir modificar la hora y volver a mostrarla. Si responde que no, el programa finaliza. Para asegurar una hora válida, las horas se aceptan de 0 a 23 y los minutos y segundos de 0 a 59.

1. ANÁLISIS DE DATOS

1.1 Entrada de datos

- Horas: número entero entre 0 y 23.
- Minutos: número entero entre 0 y 59.
- Segundos: número entero entre 0 y 59.
- Cambiar: número entero; 1 significa Sí y 0 significa No.

1.2 Proceso

- Solicitar y validar las horas hasta obtener un valor entre 0 y 23.
- Solicitar y validar los minutos y segundos entre 0 y 59.
- Mostrar la hora con dos dígitos para cada componente: HH:MM:SS.
- Preguntar si el usuario desea cambiarla y validar que la respuesta sea 0 o 1.
- Si la respuesta es 1, repetir la lectura completa de la hora.
- Si la respuesta es 0, terminar el ciclo y finalizar el programa.

1.3 Salida de datos

- Mensajes de error para horas, minutos, segundos o respuesta inválidos.
- Hora registrada en formato HH:MM:SS.
- Pregunta para cambiar la hora.
- Mensaje de finalización.

2. ALGORITMO

1. **Inicio.**
2. Declarar horas, minutos, segundos y cambiar como enteros.
3. Solicitar las horas.
4. Mientras las horas estén fuera de 0 a 23, mostrar un error y volver a solicitarlas.
5. Solicitar y validar los minutos entre 0 y 59.
6. Solicitar y validar los segundos entre 0 y 59.
7. Mostrar la hora registrada en formato HH:MM:SS.
8. Preguntar si desea cambiar la hora.
9. Mientras cambiar no sea 0 ni 1, mostrar un error y volver a solicitar la respuesta.
10. Si cambiar es 1, regresar a la lectura de horas, minutos y segundos.

11. Si cambiar es 0, mostrar que el programa ha finalizado.

12. **Fin.**

3. PSEUDOCÓDIGO

Algoritmo ControlHora

Definir horas, minutos, segundos, cambiar Como Entero;

Repetir

Repetir

 Escribir "Ingrese las horas (0 a 23):";

 Leer horas;

 Si horas < 0 O horas > 23 Entonces

 Escribir "Hora inválida.";

 FinSi;

Hasta Que horas >= 0 Y horas <= 23;

Repetir

 Escribir "Ingrese los minutos (0 a 59):";

 Leer minutos;

 Si minutos < 0 O minutos > 59 Entonces

 Escribir "Minutos inválidos.";

 FinSi;

Hasta Que minutos >= 0 Y minutos <= 59;

Repetir

 Escribir "Ingrese los segundos (0 a 59):";

 Leer segundos;

 Si segundos < 0 O segundos > 59 Entonces

 Escribir "Segundos inválidos.";

 FinSi;

Hasta Que segundos >= 0 Y segundos <= 59;

Escribir "Hora registrada: ", horas, ":", minutos, ":", segundos;

Repetir

 Escribir "¿Desea cambiar la hora? 1 Sí, 0 No:";

 Leer cambiar;

 Si cambiar <> 0 Y cambiar <> 1 Entonces

 Escribir "Respuesta inválida.";

 FinSi;

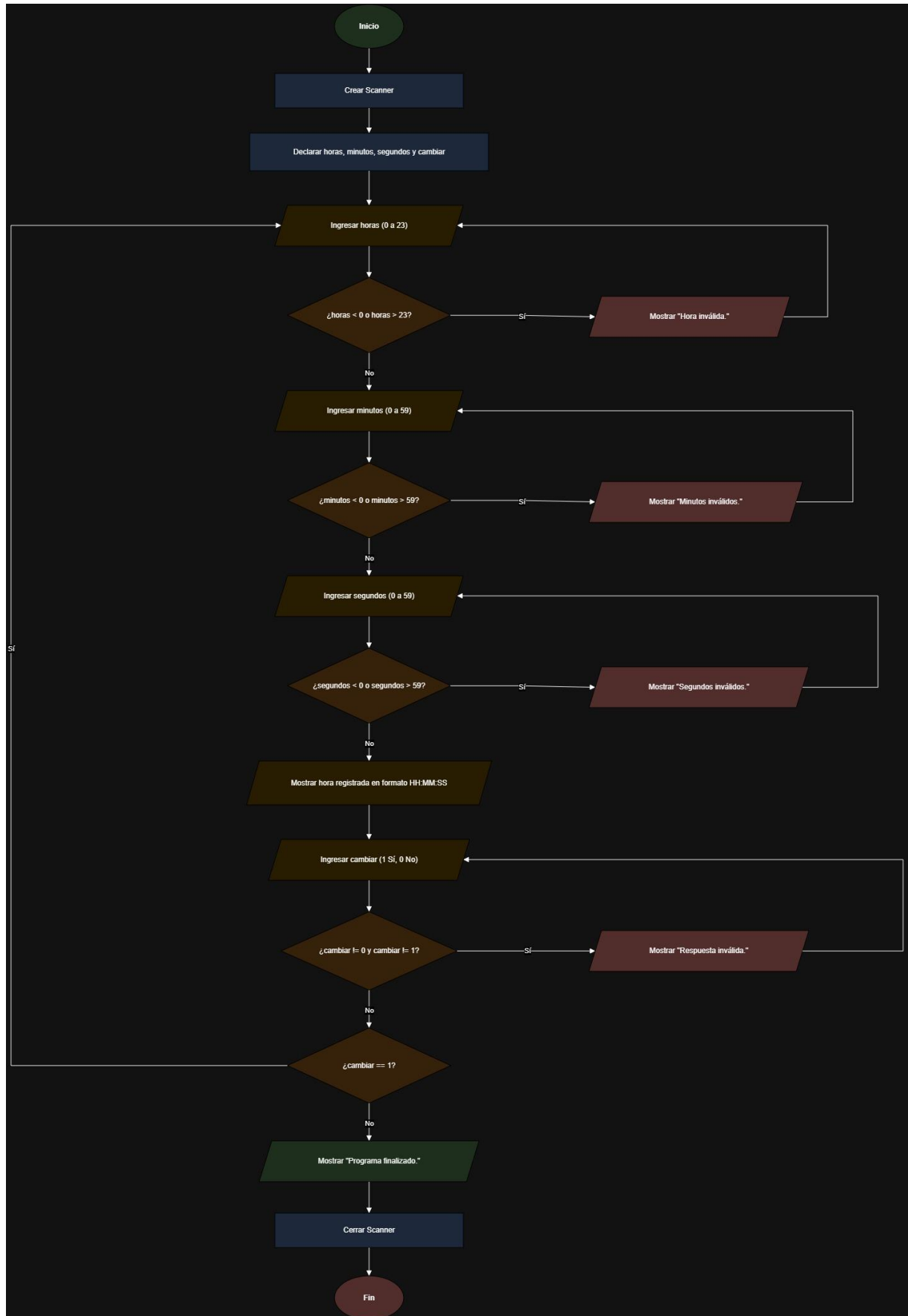
Hasta Que cambiar = 0 O cambiar = 1;

Hasta Que cambiar = 0;

Escribir "Programa finalizado.";

FinAlgoritmo

4. DIAGRAMA DE FLUJO



5. PRUEBA DE ESCRITORIO

Caso 1: Hora válida sin modificación

Variable	Valor
horas	10
minutos	30
segundos	45
hora mostrada	10:30:45
cambiar	0 - No
resultado	Programa finalizado

Operaciones:

Se validan 10 dentro de 0 a 23, 30 y 45 dentro de 0 a 59.

Hora mostrada = 10:30:45.

Como cambiar = 0, el ciclo termina.

Caso 2: Cambio de hora

Variable	Valor
primera hora	23:59:59
primera respuesta	1 - Sí
segunda hora	00:00:00
segunda respuesta	0 - No
resultado final	00:00:00

Operaciones:

23, 59 y 59 son valores válidos; se muestra 23:59:59.

Como cambiar = 1, se repite la lectura.

Se registra 0, 0, 0 y se muestra 00:00:00.

Como cambiar = 0, el programa finaliza.

Caso 3: Valores inválidos y corrección

Variable	Valor
horas iniciales	25 - inválidas
horas corregidas	14
minutos iniciales	-1 - inválidos
minutos corregidos	5
segundos iniciales	60 - inválidos
segundos corregidos	9
hora mostrada	14:05:09
cambiar	0 - No

Operaciones:

25 se rechaza y se reemplaza por 14.

-1 se rechaza y se reemplaza por 5.

60 se rechaza y se reemplaza por 9.

Resultado: 14:05:09.

Resumen de los casos

Caso	Entrada inicial	Corrección	Hora mostrada	Cambiar	Resultado
1	10, 30, 45	No aplica	10:30:45	0	Finaliza
2	23:59:59	00:00:00	Dos horas	1 → 0	Finaliza
3	25, -1, 60	14, 5, 9	14:05:09	0	Valida y finaliza

6. CÓDIGO

JAVA

```
import java.util.Scanner;
public class ControlHora {
    public static void main(String[] args) {
        Scanner entrada = new Scanner(System.in);
        int horas, minutos, segundos, cambiar;
        do {
            do {
                System.out.print("Ingrese las horas (0 a 23): ");
                horas = entrada.nextInt();
                if (horas < 0 || horas > 23)
                    System.out.println("Hora inválida.");
            } while (horas < 0 || horas > 23);
            do {
                System.out.print("Ingrese los minutos (0 a 59): ");
                minutos = entrada.nextInt();
                if (minutos < 0 || minutos > 59)
                    System.out.println("Minutos inválidos.");
            } while (minutos < 0 || minutos > 59);
            do {
                System.out.print("Ingrese los segundos (0 a 59): ");
                segundos = entrada.nextInt();
                if (segundos < 0 || segundos > 59)
                    System.out.println("Segundos inválidos.");
            } while (segundos < 0 || segundos > 59);
            System.out.printf("Hora registrada: %02d:%02d:%02d%n",
                horas, minutos, segundos);
            do {
                System.out.print("¿Desea cambiar la hora? (1 Sí, 0 No): ");
                cambiar = entrada.nextInt();
                if (cambiar != 0 && cambiar != 1)
                    System.out.println("Respuesta inválida.");
            } while (cambiar != 0 && cambiar != 1);
        } while (cambiar == 1);
        System.out.println("Programa finalizado.");
        entrada.close();
    }
}
```

C++

```
#include <iostream>
#include <iomanip>
using namespace std;
int main() {
    int horas, minutos, segundos, cambiar;
    do {
        do {
            cout << "Ingrese las horas (0 a 23): ";
            cin >> horas;
            if (horas < 0 || horas > 23) cout << "Hora invalida.\n";
        } while (horas < 0 || horas > 23);
        do {
            cout << "Ingrese los minutos (0 a 59): ";
            cin >> minutos;
            if (minutos < 0 || minutos > 59) cout << "Minutos invalidos.\n";
        } while (minutos < 0 || minutos > 59);
        do {
            cout << "Ingrese los segundos (0 a 59): ";
            cin >> segundos;
            if (segundos < 0 || segundos > 59) cout << "Segundos invalidos.\n";
        } while (segundos < 0 || segundos > 59);
        cout << "Hora registrada: " << setfill('0') << setw(2) << horas << ":"
            << setw(2) << minutos << ":" << setw(2) << segundos << "\n";
        cout << setfill(' ');
        do {
            cout << "¿Desea cambiar la hora? (1 Si, 0 No): ";
            cin >> cambiar;
            if (cambiar != 0 && cambiar != 1) cout << "Respuesta invalida.\n";
        } while (cambiar != 0 && cambiar != 1);
    } while (cambiar == 1);
    cout << "Programa finalizado.\n";
    return 0;
}
```

SISTEMA DE CUENTAS BANCARIAS

Análisis, algoritmo, pseudocódigo, flujograma, prueba de escritorio y codificación

ENUNCIADO DEL EJERCICIO

Desarrollar un programa en C++ y Java que permita registrar un cliente mediante su DNI y crear de una a tres cuentas bancarias. Para cada cuenta se registrará un número, un tipo (Ahorros o Corriente) y un saldo inicial no negativo. El usuario podrá seleccionar una cuenta y, desde un menú, ver sus atributos, enviar dinero, recibir dinero, hacer una transferencia entre sus propias cuentas, seleccionar otra cuenta o salir del sistema. Los montos de las operaciones deben ser positivos y no se permitirá enviar ni transferir una cantidad superior al saldo disponible.

1. ANÁLISIS DE DATOS

1.1 Entrada de datos

- DNI del cliente: cadena de texto obligatoria y no vacía.
- Cantidad de cuentas: número entero entre 1 y 3.
- Número de cada cuenta: cadena de texto obligatoria y no vacía.
- Tipo de cuenta: número entero; 1 para Ahorros y 2 para Corriente.
- Saldo inicial de cada cuenta: número real mayor o igual que cero.
- Cuenta seleccionada: entero entre 1 y la cantidad de cuentas creadas.
- Opción del menú: entero entre 1 y 6.
- Monto para enviar, recibir o transferir: número real positivo.
- Cuenta destino de una transferencia: cuenta existente y diferente de la cuenta de origen.

1.2 Proceso

- Registrar y validar el DNI del cliente.
- Solicitar y validar la cantidad de cuentas, con un máximo de tres.
- Repetir la creación de cuentas; validar número, tipo y saldo inicial, y guardar cada registro en variables individuales.
- Seleccionar una cuenta existente y cargar su número, tipo y saldo como cuenta actual.
- Repetir el menú hasta que se elija salir.
- En la opción 1, mostrar DNI, número, tipo y saldo de la cuenta seleccionada.
- En la opción 2, validar un monto positivo que no supere el saldo; luego restarlo y actualizar la cuenta.
- En la opción 3, validar un monto positivo; luego sumarlo y actualizar la cuenta.
- En la opción 4, comprobar que exista otra cuenta, validar el destino y el monto, restar al origen y sumar al destino.
- En la opción 5, validar y cambiar la cuenta seleccionada.
- En la opción 6, finalizar; cualquier otra opción genera un mensaje de error.

1.3 Salida de datos

- Mensajes de validación para datos incorrectos.

- Atributos de la cuenta seleccionada: DNI, número, tipo y saldo.
- Confirmación y saldo actualizado después de enviar o recibir dinero.
- Confirmación y saldo de la cuenta de origen después de una transferencia.
- Aviso cuando no existe otra cuenta para transferir.
- Mensaje de opción inválida y mensaje de finalización.

2. ALGORITMO

1. **Inicio.**
2. Declarar las variables del cliente, las tres posibles cuentas, tipos, saldos, opciones y montos.
3. Solicitar el DNI y repetir la lectura mientras esté vacío.
4. Solicitar cuántas cuentas se crearán y validar que la cantidad esté entre 1 y 3.
5. Para cada cuenta, solicitar y validar el número, el tipo y el saldo inicial.
6. Guardar la información en las variables correspondientes a la cuenta 1, 2 o 3.
7. Solicitar y validar la cuenta que quedará seleccionada.
8. Cargar en las variables actuales el número, tipo y saldo de la cuenta seleccionada.
9. Mostrar el menú y leer la opción.
10. Si la opción es 1, mostrar los atributos de la cuenta actual.
11. Si la opción es 2, solicitar un monto positivo que no exceda el saldo, restarlo y actualizar la cuenta.
12. Si la opción es 3, solicitar un monto positivo, sumarlo y actualizar la cuenta.
13. Si la opción es 4 y solo existe una cuenta, indicar que no puede transferirse.
14. Si existen varias cuentas, validar una cuenta destino diferente y un monto disponible; restarlo al origen y sumarlo al destino.
15. Si la opción es 5, solicitar y validar otra cuenta seleccionada.
16. Si la opción no pertenece al menú, mostrar “Opción inválida”.
17. Repetir el menú mientras la opción sea diferente de 6.
18. Mostrar “Sistema finalizado”.
19. **Fin.**

3. PSEUDOCÓDIGO

Algoritmo SistemaCuentasBancarias

```
Definir dni, numeroCuenta, tipoCuenta, numeroActual, tipoActual Como Cadena;
Definir cuenta1, cuenta2, cuenta3, tipo1, tipo2, tipo3 Como Cadena;
Definir cantidadCuentas, i, tipo, cuentaSeleccionada, destino, opcion Como Entero;
Definir saldoInicial, saldoActual, saldoDestino, monto, saldo1, saldo2, saldo3 Como Real;
cuenta1 <- ""; cuenta2 <- ""; cuenta3 <- "";
saldo1 <- 0; saldo2 <- 0; saldo3 <- 0;
Repetir
    Escribir "Ingrese el DNI del cliente:";
    Leer dni;
Hasta Que Longitud(dni) > 0;
Repetir
    Escribir "¿Cuántas cuentas desea crear? (1 a 3):";
    Leer cantidadCuentas;
Hasta Que cantidadCuentas >= 1 Y cantidadCuentas <= 3;
Para i <- 1 Hasta cantidadCuentas Hacer
    Repetir
        Escribir "Número de la cuenta ", i, ":";
        Leer numeroCuenta;
Hasta Que Longitud(numeroCuenta) > 0;
    Repetir
        Escribir "Tipo (1 Ahorros, 2 Corriente):";
        Leer tipo;
Hasta Que tipo = 1 O tipo = 2;
    Si tipo = 1 Entonces
        tipoCuenta <- "Ahorros";
    SiNo
        tipoCuenta <- "Corriente";
    FinSi;
    Repetir
        Escribir "Saldo inicial:";
        Leer saldoInicial;
Hasta Que saldoInicial >= 0;
    Segun i Hacer
        1: cuenta1 <- numeroCuenta; tipo1 <- tipoCuenta; saldo1 <- saldoInicial;
        2: cuenta2 <- numeroCuenta; tipo2 <- tipoCuenta; saldo2 <- saldoInicial;
        3: cuenta3 <- numeroCuenta; tipo3 <- tipoCuenta; saldo3 <- saldoInicial;
    FinSegun;
FinPara;
Repetir
    Escribir "Seleccione una cuenta (1 a ", cantidadCuentas, "):";
    Leer cuentaSeleccionada;
Hasta Que cuentaSeleccionada >= 1 Y cuentaSeleccionada <= cantidadCuentas;
Repetir
    Segun cuentaSeleccionada Hacer
        1: numeroActual <- cuenta1; tipoActual <- tipo1; saldoActual <- saldo1;
        2: numeroActual <- cuenta2; tipoActual <- tipo2; saldoActual <- saldo2;
        3: numeroActual <- cuenta3; tipoActual <- tipo3; saldoActual <- saldo3;
```

```

FinSegun;
Escribir "1. Ver atributos 2. Enviar 3. Recibir";
Escribir "4. Transferir 5. Cambiar cuenta 6. Salir";
Leer opcion;
Segun opcion Hacer
  1:
    Escribir dni, numeroActual, tipoActual, saldoActual;
  2:
    Repetir
      Escribir "Monto a enviar:"; Leer monto;
      Hasta Que monto > 0 Y monto <= saldoActual;
      saldoActual <- saldoActual - monto;
  3:
    Repetir
      Escribir "Monto a recibir:"; Leer monto;
      Hasta Que monto > 0;
      saldoActual <- saldoActual + monto;
  4:
    Si cantidadCuentas = 1 Entonces
      Escribir "No existe otra cuenta para transferir.";
    SiNo
      Repetir
        Escribir "Cuenta destino:"; Leer destino;
        Hasta Que destino >= 1 Y destino <= cantidadCuentas Y destino <>
          cuentaSeleccionada;
      Repetir
        Escribir "Monto a transferir:"; Leer monto;
        Hasta Que monto > 0 Y monto <= saldoActual;
        Si destino = 1 Entonces saldoDestino <- saldo1;
        Si destino = 2 Entonces saldoDestino <- saldo2;
        Si destino = 3 Entonces saldoDestino <- saldo3;
        saldoActual <- saldoActual - monto;
        saldoDestino <- saldoDestino + monto;
        Si destino = 1 Entonces saldo1 <- saldoDestino;
        Si destino = 2 Entonces saldo2 <- saldoDestino;
        Si destino = 3 Entonces saldo3 <- saldoDestino;
      FinSi;
  5:
    Repetir
      Escribir "Seleccione otra cuenta:"; Leer cuentaSeleccionada;
      Hasta Que cuentaSeleccionada >= 1 Y cuentaSeleccionada <= cantidadCuentas;
  6:
    Escribir "Sistema finalizado.";
De Otro Modo:
    Escribir "Opción inválida.";
FinSegun;
Si opcion = 2 O opcion = 3 O opcion = 4 Entonces
  Si cuentaSeleccionada = 1 Entonces saldo1 <- saldoActual;
  Si cuentaSeleccionada = 2 Entonces saldo2 <- saldoActual;

```

```
        Si cuentaSeleccionada = 3 Entonces saldo3 <- saldoActual;  
    FinSi;  
    Hasta Que opcion = 6;  
FinAlgoritmo
```


4. DIAGRAMA DE FLUJO



5. PRUEBA DE ESCRITORIO

Caso 1: Recibir dinero en una cuenta de Ahorros

Variable	Valor
DNI	0102030405
cantidadCuentas	1
cuenta1 / tipo1	AH-001 / Ahorros
saldo1 inicial	\$100,00
cuentaSeleccionada	1
opción	3 - Recibir dinero
monto	\$50,00
saldo1 final	\$150,00
resultado	Dinero recibido

Operaciones:

Validación: cantidadCuentas = 1 está entre 1 y 3.

Validación: monto = 50,00 > 0.

saldoActual = 100,00 + 50,00 = 150,00.

saldo1 se actualiza con \$150,00.

Caso 2: Envío inválido y corrección

Variable	Valor
DNI	1122334455
cantidadCuentas	2
cuenta seleccionada	2 - CR-200
tipo / saldo inicial	Corriente / \$200,00
primer monto	\$250,00 - inválido
monto corregido	\$50,00
saldo final	\$150,00
resultado	Envío realizado

Operaciones:

250,00 > 200,00; el primer monto se rechaza por saldo insuficiente.

50,00 > 0 y 50,00 <= 200,00; el monto corregido es válido.

saldoActual = 200,00 - 50,00 = 150,00.

saldo2 se actualiza con \$150,00.

Caso 3: Transferencia entre cuentas

Variable	Valor
DNI	9988776655
cantidadCuentas	3

Variable	Valor
origen	Cuenta 1 - saldo \$300,00
destino	Cuenta 2 - saldo \$100,00
monto	\$75,00
saldo origen final	\$225,00
saldo destino final	\$175,00
resultado	Transferencia realizada

Operaciones:

El destino 2 existe y es diferente del origen 1.

$75,00 > 0$ y $75,00 \leq 300,00$; el monto es válido.

$\text{saldo1} = 300,00 - 75,00 = 225,00$.

$\text{saldo2} = 100,00 + 75,00 = 175,00$.

Resumen de los casos

Caso	Operación	Saldo inicial	Monto	Saldo final	Resultado
1	Recibir	\$100,00	\$50,00	\$150,00	Aceptado
2	Enviar	\$200,00	$\$250,00 \rightarrow \$50,00$	\$150,00	Valida saldo
3	Transferir	$\$300,00 / \$100,00$	\$75,00	$\$225,00 / \$175,00$	Aceptado

6. CÓDIGO

JAVA

```
import java.util.Scanner;
public class SistemaCuentasBancarias {
    public static void main(String[] args) {
        Scanner entrada = new Scanner(System.in);
        String dni, numeroCuenta, tipoCuenta, numeroActual = "", tipoActual = "";
        String cuenta1 = "", cuenta2 = "", cuenta3 = "";
        String tipo1 = "", tipo2 = "", tipo3 = "";
        int cantidadCuentas, i, tipo, cuentaSeleccionada, destino, opcion;
        double saldoInicial, saldoActual = 0, saldoDestino = 0, monto;
        double saldo1 = 0, saldo2 = 0, saldo3 = 0;
        // Primero se registran los datos generales del cliente.
        System.out.print("Ingrese el DNI del cliente: ");
        dni = entrada.nextLine();
        while (dni.trim().isEmpty()) {
            System.out.print("El DNI no puede estar vacío. Ingrese nuevamente: ");
            dni = entrada.nextLine();
        }
        do {
            System.out.print("¿Cuántas cuentas desea crear? (1 a 3): ");
            cantidadCuentas = entrada.nextInt();
            if (cantidadCuentas < 1 || cantidadCuentas > 3)
                System.out.println("Cantidad de cuentas inválida.");
        } while (cantidadCuentas < 1 || cantidadCuentas > 3);
        // Se usa un for para crear solamente las cuentas solicitadas.
        for (i = 1; i <= cantidadCuentas; i++) {
            entrada.nextLine();
            System.out.print("Número de la cuenta " + i + ": ");
            numeroCuenta = entrada.nextLine();
            while (numeroCuenta.trim().isEmpty()) {
                System.out.print("El número no puede estar vacío. Ingrese nuevamente: ");
                numeroCuenta = entrada.nextLine();
            }
            do {
                System.out.print("Tipo (1 Ahorros, 2 Corriente): ");
                tipo = entrada.nextInt();
                if (tipo != 1 && tipo != 2) System.out.println("Tipo inválido.");
            } while (tipo != 1 && tipo != 2);
            tipoCuenta = tipo == 1 ? "Ahorros" : "Corriente";
            do {
                System.out.print("Saldo inicial: $ ");
                saldoInicial = entrada.nextDouble();
                if (saldoInicial < 0) System.out.println("El saldo no puede ser negativo.");
            } while (saldoInicial < 0);
            // Cada cuenta se guarda en variables simples, sin usar arreglos.
            switch (i) {
                case 1: cuenta1 = numeroCuenta; tipo1 = tipoCuenta; saldo1 = saldoInicial; break;
                case 2: cuenta2 = numeroCuenta; tipo2 = tipoCuenta; saldo2 = saldoInicial; break;
                case 3: cuenta3 = numeroCuenta; tipo3 = tipoCuenta; saldo3 = saldoInicial; break;
            }
        }
        // El menú se repite hasta que el usuario seleccione salir.
        do {
            System.out.print("Seleccione una cuenta (1 a " + cantidadCuentas + "): ");
            cuentaSeleccionada = entrada.nextInt();
        } while (cuentaSeleccionada < 1 || cuentaSeleccionada > cantidadCuentas);
        do {
            switch (cuentaSeleccionada) {
                case 1: numeroActual = cuenta1; tipoActual = tipo1; saldoActual = saldo1; break;
```

```

        case 2: numeroActual = cuenta2; tipoActual = tipo2; saldoActual = saldo2; break;
        case 3: numeroActual = cuenta3; tipoActual = tipo3; saldoActual = saldo3; break;
    }
    System.out.println("\n--- MENÚ BANCARIO ---");
    System.out.println("Cuenta seleccionada: " + numeroActual);
    System.out.println("1. Ver atributos de la cuenta");
    System.out.println("2. Enviar dinero");
    System.out.println("3. Recibir dinero");
    System.out.println("4. Transferir entre cuentas");
    System.out.println("5. Seleccionar otra cuenta");
    System.out.println("6. Salir");
    System.out.print("Seleccione una opción: ");
    opcion = entrada.nextInt();
    switch (opcion) {
        case 1:
            System.out.println("DNI: " + dni);
            System.out.println("Número: " + numeroActual);
            System.out.println("Tipo: " + tipoActual);
            System.out.printf("Saldo: $%.2f%n", saldoActual);
            break;
        case 2:
            do {
                System.out.print("Monto a enviar: $ ");
                monto = entrada.nextDouble();
                if (monto <= 0 || monto > saldoActual)
                    System.out.println("Monto inválido o saldo insuficiente.");
            } while (monto <= 0 || monto > saldoActual);
            saldoActual -= monto;
            if (cuentaSeleccionada == 1) saldo1 = saldoActual;
            else if (cuentaSeleccionada == 2) saldo2 = saldoActual;
            else saldo3 = saldoActual;
            System.out.printf("Envío realizado. Saldo: $%.2f%n", saldoActual);
            break;
        case 3:
            do {
                System.out.print("Monto a recibir: $ ");
                monto = entrada.nextDouble();
                if (monto <= 0) System.out.println("El monto debe ser positivo.");
            } while (monto <= 0);
            saldoActual += monto;
            if (cuentaSeleccionada == 1) saldo1 = saldoActual;
            else if (cuentaSeleccionada == 2) saldo2 = saldoActual;
            else saldo3 = saldoActual;
            System.out.printf("Dinero recibido. Saldo: $%.2f%n", saldoActual);
            break;
        case 4:
            // La transferencia modifica el saldo de dos cuentas del cliente.
            if (cantidadCuentas == 1) {
                System.out.println("No existe otra cuenta para transferir.");
            } else {
                do {
                    System.out.print("Cuenta destino (1 a " + cantidadCuentas + "): ");
                    destino = entrada.nextInt();
                    if (destino < 1 || destino > cantidadCuentas || destino == cuentaSeleccionada)
                        System.out.println("Cuenta destino inválida.");
                } while (destino < 1 || destino > cantidadCuentas || destino ==
cuentaSeleccionada);
                do {
                    System.out.print("Monto a transferir: $ ");
                    monto = entrada.nextDouble();
                    if (monto <= 0 || monto > saldoActual)

```

```

        System.out.println("Monto inválido o saldo insuficiente.");
    } while (monto <= 0 || monto > saldoActual);
    if (destino == 1) saldoDestino = saldo1;
    else if (destino == 2) saldoDestino = saldo2;
    else saldoDestino = saldo3;
    saldoActual -= monto;
    saldoDestino += monto;
    if (cuentaSeleccionada == 1) saldo1 = saldoActual;
    else if (cuentaSeleccionada == 2) saldo2 = saldoActual;
    else saldo3 = saldoActual;
    if (destino == 1) saldo1 = saldoDestino;
    else if (destino == 2) saldo2 = saldoDestino;
    else saldo3 = saldoDestino;
    System.out.printf("Transferencia realizada. Saldo origen: $%.2f%n",
saldoActual);
    }
    break;
case 5:
    do {
        System.out.print("Seleccione una cuenta (1 a " + cantidadCuentas + "): ");
        cuentaSeleccionada = entrada.nextInt();
    } while (cuentaSeleccionada < 1 || cuentaSeleccionada > cantidadCuentas);
    break;
case 6:
    System.out.println("Sistema finalizado.");
    break;
default:
    System.out.println("Opción inválida.");
    }
} while (opcion != 6);
entrada.close();
}
}

```

C++

```
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;
int main() {
    string dni, numeroCuenta, tipoCuenta, numeroActual, tipoActual;
    string cuenta1 = "", cuenta2 = "", cuenta3 = "";
    string tipo1 = "", tipo2 = "", tipo3 = "";
    int cantidadCuentas, i, tipo, cuentaSeleccionada, destino, opcion;
    double saldoInicial, saldoActual = 0, saldoDestino = 0, monto;
    double saldo1 = 0, saldo2 = 0, saldo3 = 0;
    // Se registra el DNI y se valida que no quede vacio.
    cout << "Ingrese el DNI del cliente: "; getline(cin, dni);
    while (dni.empty()) { cout << "DNI vacio. Ingrese nuevamente: "; getline(cin, dni); }
    do {
        cout << "¿Cuántas cuentas desea crear? (1 a 3): "; cin >> cantidadCuentas;
    } while (cantidadCuentas < 1 || cantidadCuentas > 3);
    // Se crean hasta tres cuentas usando variables simples, sin arreglos.
    for (i = 1; i <= cantidadCuentas; i++) {
        cin.ignore(10000, '\n');
        cout << "Numero de la cuenta " << i << ": "; getline(cin, numeroCuenta);
        while (numeroCuenta.empty()) { cout << "Numero vacio. Ingrese nuevamente: ";
        getline(cin, numeroCuenta); }
        do { cout << "Tipo (1 Ahorros, 2 Corriente): "; cin >> tipo; }
        while (tipo != 1 && tipo != 2);
        tipoCuenta = tipo == 1 ? "Ahorros" : "Corriente";
        do { cout << "Saldo inicial: $ "; cin >> saldoInicial; }
        while (saldoInicial < 0);
        switch (i) {
            case 1: cuenta1 = numeroCuenta; tipo1 = tipoCuenta; saldo1 = saldoInicial; break;
            case 2: cuenta2 = numeroCuenta; tipo2 = tipoCuenta; saldo2 = saldoInicial; break;
            case 3: cuenta3 = numeroCuenta; tipo3 = tipoCuenta; saldo3 = saldoInicial; break;
        }
    }
    do { cout << "Seleccione una cuenta (1 a " << cantidadCuentas << "): "; cin >>
    cuentaSeleccionada; }
    while (cuentaSeleccionada < 1 || cuentaSeleccionada > cantidadCuentas);
    // El menu se repite hasta que se elija la opcion 6.
    do {
        switch (cuentaSeleccionada) {
            case 1: numeroActual = cuenta1; tipoActual = tipo1; saldoActual = saldo1; break;
            case 2: numeroActual = cuenta2; tipoActual = tipo2; saldoActual = saldo2; break;
            case 3: numeroActual = cuenta3; tipoActual = tipo3; saldoActual = saldo3; break;
        }
        cout << "\n--- MENU BANCARIO ---\n";
        cout << "Cuenta seleccionada: " << numeroActual << "\n";
        cout << "1. Ver atributos\n2. Enviar dinero\n3. Recibir dinero\n";
        cout << "4. Transferir entre cuentas\n5. Seleccionar otra cuenta\n6. Salir\n";
        cout << "Opcion: ";
        cin >> opcion;
        switch (opcion) {
            case 1:
                cout << "DNI: " << dni << "\n";
                cout << "Numero: " << numeroActual << "\n";
                cout << "Tipo: " << tipoActual;
                cout << fixed << setprecision(2) << "\nSaldo: $" << saldoActual << "\n";
                break;
            case 2:
                do { cout << "Monto a enviar: $ "; cin >> monto; }
                while (monto <= 0 || monto > saldoActual);
                saldoActual -= monto;
                if (cuentaSeleccionada == 1) saldo1 = saldoActual;
                else if (cuentaSeleccionada == 2) saldo2 = saldoActual;
```

```

else saldo3 = saldoActual;
cout << "Envio realizado. Saldo: $" << saldoActual << "\n";
break;
case 3:
do { cout << "Monto a recibir: $ "; cin >> monto; } while (monto <= 0);
saldoActual += monto;
if (cuentaSeleccionada == 1) saldo1 = saldoActual;
else if (cuentaSeleccionada == 2) saldo2 = saldoActual;
else saldo3 = saldoActual;
cout << "Dinero recibido. Saldo: $" << saldoActual << "\n";
break;
case 4:
// Se resta al origen y se suma el mismo monto al destino.
if (cantidadCuentas == 1) cout << "No existe otra cuenta para transferir.\n";
else {
do { cout << "Cuenta destino: "; cin >> destino; }
while (destino < 1 || destino > cantidadCuentas || destino == cuentaSeleccionada);
do { cout << "Monto a transferir: $ "; cin >> monto; }
while (monto <= 0 || monto > saldoActual);
if (destino == 1) saldoDestino = saldo1;
else if (destino == 2) saldoDestino = saldo2;
else saldoDestino = saldo3;
saldoActual -= monto; saldoDestino += monto;
if (cuentaSeleccionada == 1) saldo1 = saldoActual;
else if (cuentaSeleccionada == 2) saldo2 = saldoActual;
else saldo3 = saldoActual;
if (destino == 1) saldo1 = saldoDestino;
else if (destino == 2) saldo2 = saldoDestino;
else saldo3 = saldoDestino;
cout << "Transferencia realizada. Saldo origen: $" << saldoActual << "\n";
}
break;
case 5:
do { cout << "Seleccione una cuenta: "; cin >> cuentaSeleccionada; }
while (cuentaSeleccionada < 1 || cuentaSeleccionada > cantidadCuentas);
break;
case 6: cout << "Sistema finalizado.\n"; break;
default: cout << "Opcion invalida.\n";
}
} while (opcion != 6);
return 0;
}

```


GESTIÓN DE UN JEAN EN FÁBRICA O TIENDA

Análisis, algoritmo, pseudocódigo, flujograma, prueba de escritorio y codificación

ENUNCIADO DEL EJERCICIO

Desarrollar un programa en C++ y Java que permita registrar los datos de un jean: código, color, talla, si fue teñido, cantidad de teñidos, precio, cantidad de botones, humedad y estado de la tela. El sistema debe tener opciones para mostrar los datos del jean, lavarlo, secarlo y salir. Al lavar, disminuye la cantidad de teñidos; al secar, disminuye la humedad. Se deben utilizar estructuras de selección. Toda la lógica se desarrollará de forma directa, sin métodos adicionales ni una clase específica para representar el jean.

1. ANÁLISIS DE DATOS

1.1 Entrada de datos

- Código del jean: cadena obligatoria y no vacía.
- Color: cadena obligatoria y no vacía.
- Talla: cadena obligatoria y no vacía.
- Fue teñido: entero; 1 representa Sí y 0 representa No.
- Cantidad de teñidos: entero mayor que cero si fue teñido; de lo contrario se asigna cero.
- Precio: número real mayor o igual que cero.
- Cantidad de botones: entero mayor o igual que cero.
- Humedad: número real entre 0 y 100.
- Estado de la tela: entero; 1 Buena, 2 Regular o 3 Dañada.
- Opción: entero; 1 Mostrar datos, 2 Lavar, 3 Secar o 4 Salir.

1.2 Proceso

- Registrar y validar los datos mediante variables simples.
- Usar if-else para determinar si se solicita la cantidad de teñidos.
- Usar if-else para convertir el estado seleccionado en Buena, Regular o Dañada.
- Repetir el menú con do-while hasta elegir salir.
- Usar switch para ejecutar Mostrar datos, Lavar, Secar o Salir.
- Al mostrar, presentar directamente el contenido de todas las variables.
- Al lavar, usar if-else para comprobar si quedan teñidos; disminuir uno y cambiar fue teñido a No al llegar a cero.
- Al secar, usar if-else para comprobar la humedad; restar 10 y evitar que el resultado sea negativo.
- Mostrar un error cuando la opción no corresponda al menú.

1.3 Salida de datos

- Datos completos y actualizados del jean.
- Cantidad de teñidos después de lavar.

- Humedad después de secar.
- Mensajes para datos u opciones inválidas.
- Avisos cuando no queden teñidos o cuando el jean ya esté seco.
- Mensaje de finalización.

2. ALGORITMO

1. **Inicio.**
2. Declarar las variables para los datos del jean y el menú.
3. Solicitar y validar el código, color y talla.
4. Preguntar si el jean fue teñido y aceptar solamente 0 o 1.
5. Si fue teñido, solicitar una cantidad mayor que cero; en caso contrario, asignar cero.
6. Solicitar y validar el precio, la cantidad de botones y la humedad.
7. Solicitar el estado de la tela y asignar Buena, Regular o Dañada mediante if-else.
8. Mostrar el menú y leer una opción.
9. Si la opción es 1, mostrar directamente todos los datos registrados.
10. Si la opción es 2 y quedan teñidos, disminuir uno; si llega a cero, indicar que ya no está teñido.
11. Si la opción es 2 y no quedan teñidos, mostrar el aviso correspondiente.
12. Si la opción es 3 y la humedad es mayor que cero, disminuirla en 10 sin bajar de cero.
13. Si la opción es 3 y la humedad es cero, indicar que el jean ya está seco.
14. Si la opción es 4, mostrar el mensaje de finalización.
15. Si la opción no es válida, mostrar un error.
16. Repetir el menú mientras la opción sea diferente de 4.
17. **Fin.**

3. PSEUDOCÓDIGO

Algoritmo GestionJean

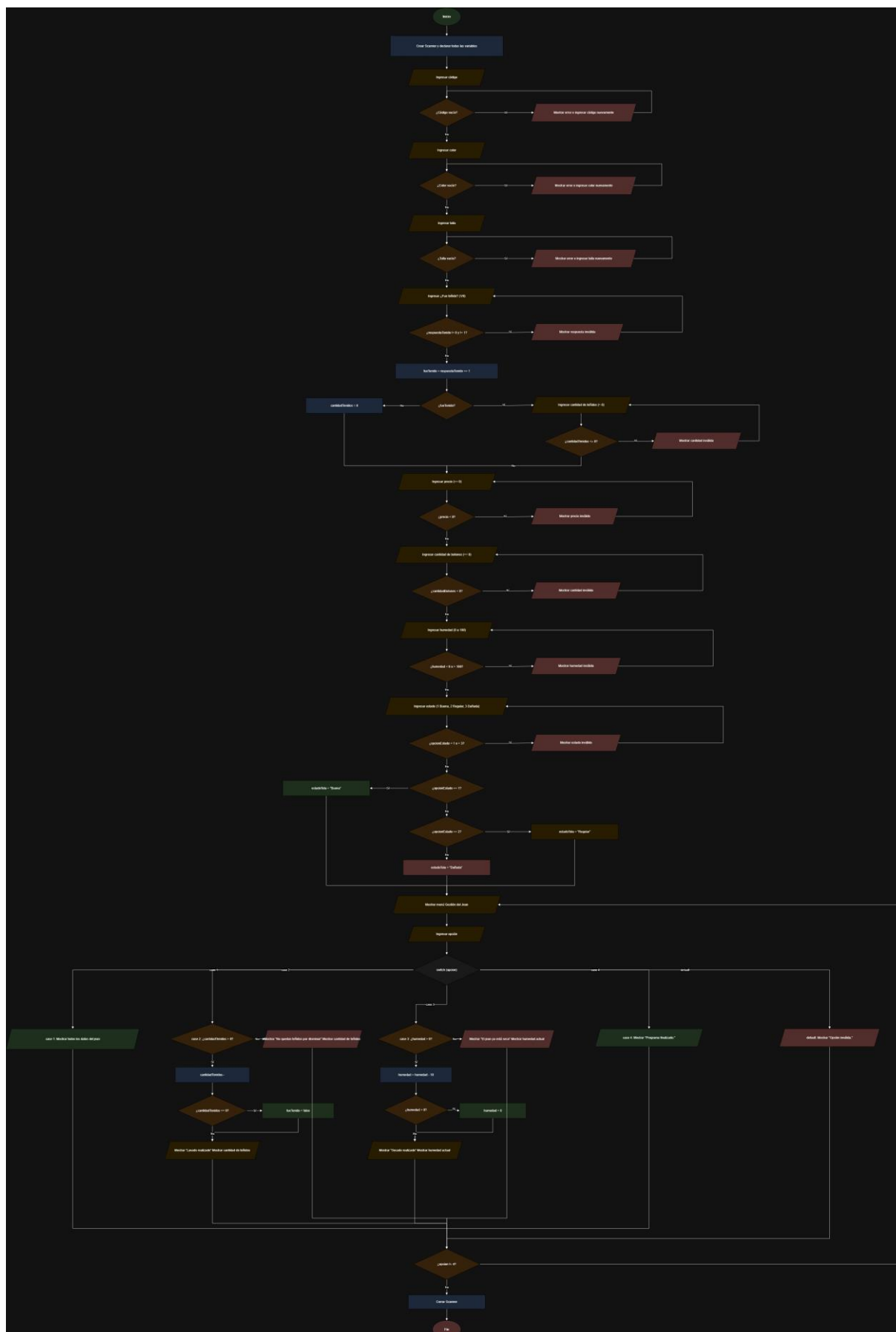
```
Definir codigo, color, talla, estadoTela Como Cadena;
Definir respuestaTenido, cantidadTenidos, cantidadBotones Como Entero;
Definir opcionEstado, opcion Como Entero;
Definir precio, humedad Como Real;
Definir fueTenido Como Logico;
Repetir
    Escribir "Código del jean:";
    Leer codigo;
Hasta Que Longitud(codigo) > 0;
Repetir
    Escribir "Color:";
    Leer color;
Hasta Que Longitud(color) > 0;
Repetir
    Escribir "Talla:";
    Leer talla;
Hasta Que Longitud(talla) > 0;
Repetir
    Escribir "¿Fue teñido? 1 Sí, 0 No:";
    Leer respuestaTenido;
Hasta Que respuestaTenido = 0 O respuestaTenido = 1;
fueTenido <- respuestaTenido = 1;
Si fueTenido Entonces
    Repetir
        Escribir "Cantidad de teñidos:";
        Leer cantidadTenidos;
        Hasta Que cantidadTenidos > 0;
SiNo
    cantidadTenidos <- 0;
FinSi;
Repetir
    Escribir "Precio:";
    Leer precio;
Hasta Que precio >= 0;
Repetir
    Escribir "Cantidad de botones:";
    Leer cantidadBotones;
Hasta Que cantidadBotones >= 0;
Repetir
    Escribir "Humedad entre 0 y 100:";
    Leer humedad;
Hasta Que humedad >= 0 Y humedad <= 100;
Repetir
    Escribir "Estado: 1 Buena, 2 Regular, 3 Dañada:";
    Leer opcionEstado;
Hasta Que opcionEstado >= 1 Y opcionEstado <= 3;
Si opcionEstado = 1 Entonces
```

```

    estadoTela <- "Buena";
SiNo
    Si opcionEstado = 2 Entonces
        estadoTela <- "Regular";
    SiNo
        estadoTela <- "Dañada";
    FinSi;
FinSi;
Repetir
    Escribir "1 Mostrar 2 Lavar 3 Secar 4 Salir";
    Leer opcion;
    Segun opcion Hacer
        1:
            Escribir codigo, color, talla, fueTenido, cantidadTenidos;
            Escribir precio, cantidadBotones, humedad, estadoTela;
        2:
            Si cantidadTenidos > 0 Entonces
                cantidadTenidos <- cantidadTenidos - 1;
            Si cantidadTenidos = 0 Entonces
                fueTenido <- Falso;
            FinSi;
            Escribir "Lavado realizado.";
        SiNo
            Escribir "No quedan teñidos por disminuir.";
        FinSi;
        Escribir "Cantidad de teñidos: ", cantidadTenidos;
    3:
        Si humedad > 0 Entonces
            humedad <- humedad - 10;
        Si humedad < 0 Entonces
            humedad <- 0;
        FinSi;
        Escribir "Secado realizado.";
        SiNo
            Escribir "El jean ya está seco.";
        FinSi;
        Escribir "Humedad actual: ", humedad, "%";
    4:
        Escribir "Programa finalizado.";
    De Otro Modo:
        Escribir "Opción inválida.";
    FinSegun;
Hasta Que opcion = 4;
FinAlgoritmo

```

4. DIAGRAMA DE FLUJO



5. PRUEBA DE ESCRITORIO

Caso 1: Mostrar los datos registrados

Variable	Valor
código	JN-001
color	Azul
talla	32
fue teñido	Sí
cantidad de teñidos	3
precio	\$35,50
botones	5
humedad	40 %
estado	Buena
opción	1 - Mostrar

Operaciones:

Las entradas cumplen los rangos establecidos.

La opción 1 muestra las variables sin modificar sus valores.

Resultado: se presentan todos los datos registrados.

Caso 2: Lavar un jean teñido

Variable	Valor
código	JN-002
fue teñido inicial	Sí
cantidad inicial	2
opción	2 - Lavar
operación	2 - 1
cantidad final	1
fue teñido final	Sí
resultado	Lavado realizado

Operaciones:

$\text{cantidadTenidos} = 2 > 0$; se ejecuta la rama verdadera del if.

$\text{cantidadTenidos} = 2 - 1 = 1$.

Como el valor final no es cero, fueTenido permanece Verdadero.

Caso 3: Secar sin obtener humedad negativa

Variable	Valor
código	JN-003
humedad inicial	5 %
opción	3 - Secar

Variable	Valor
resta	$5 - 10 = -5$
corrección	-5 se cambia a 0
humedad final	0 %
segunda acción	Secar nuevamente
resultado	El jean ya está seco

Operaciones:

humedad = $5 > 0$; se ejecuta la rama de secado.

humedad = $5 - 10 = -5$.

Como humedad < 0 , se asigna humedad = 0.

Al secar nuevamente, se ejecuta el else y se conserva 0 %.

Resumen de los casos

Caso	Opción	Valor inicial	Operación	Valor final	Resultado
1	Mostrar	Datos completos	Sin cambios	Datos iguales	Muestra
2	Lavar	2 teñidos	$2 - 1$	1 teñido	Actualiza
3	Secar	5 %	$5 - 10$; mínimo 0	0 %	Jean seco

6. CÓDIGO

JAVA

```
import java.util.Scanner;
public class GestionJean {
    public static void main(String[] args) {
        Scanner entrada = new Scanner(System.in);
        String codigo, color, talla, estadoTela;
        int respuestaTenido, cantidadTenidos, cantidadBotones, opcionEstado, opcion;
        double precio, humedad;
        boolean fueTenido;
        // Se registran y validan los datos del jean.
        System.out.print("Código del jean: ");
        codigo = entrada.nextLine();
        while (codigo.trim().isEmpty()) {
            System.out.print("El código no puede estar vacío. Ingrese nuevamente: ");
            codigo = entrada.nextLine();
        }
        System.out.print("Color: ");
        color = entrada.nextLine();
        while (color.trim().isEmpty()) {
            System.out.print("El color no puede estar vacío. Ingrese nuevamente: ");
            color = entrada.nextLine();
        }
        System.out.print("Talla: ");
        talla = entrada.nextLine();
        while (talla.trim().isEmpty()) {
            System.out.print("La talla no puede estar vacía. Ingrese nuevamente: ");
            talla = entrada.nextLine();
        }
        do {
            System.out.print("¿Fue teñido? (1 Sí, 0 No): ");
            respuestaTenido = entrada.nextInt();
            if (respuestaTenido != 0 && respuestaTenido != 1)
                System.out.println("Respuesta inválida.");
        } while (respuestaTenido != 0 && respuestaTenido != 1);
        fueTenido = respuestaTenido == 1;
        if (fueTenido) {
            do {
                System.out.print("Cantidad de teñidos (mayor que 0): ");
                cantidadTenidos = entrada.nextInt();
                if (cantidadTenidos <= 0) System.out.println("Cantidad inválida.");
            } while (cantidadTenidos <= 0);
        } else cantidadTenidos = 0;
        do {
            System.out.print("Precio (mayor o igual que 0): $ ");
            precio = entrada.nextDouble();
            if (precio < 0) System.out.println("Precio inválido.");
        } while (precio < 0);
        do {
            System.out.print("Cantidad de botones (0 o más): ");
            cantidadBotones = entrada.nextInt();
            if (cantidadBotones < 0) System.out.println("Cantidad inválida.");
        } while (cantidadBotones < 0);
        do {
            System.out.print("Humedad del jean (0 a 100): ");
            humedad = entrada.nextDouble();
            if (humedad < 0 || humedad > 100) System.out.println("Humedad inválida.");
        } while (humedad < 0 || humedad > 100);
        do {
            System.out.print("Estado de la tela (1 Buena, 2 Regular, 3 Dañada): ");
            opcionEstado = entrada.nextInt();
            if (opcionEstado < 1 || opcionEstado > 3) System.out.println("Estado inválido.");
        } while (opcionEstado < 1 || opcionEstado > 3);
        if (opcionEstado == 1) estadoTela = "Buena";
        else if (opcionEstado == 2) estadoTela = "Regular";
    }
}
```



```

else estadoTela = "Dañada";
// Toda la lógica está dentro de main, sin métodos adicionales.
do {
    System.out.println("\n--- GESTIÓN DEL JEAN ---");
    System.out.println("1. Mostrar datos");
    System.out.println("2. Lavar jean");
    System.out.println("3. Secar jean");
    System.out.println("4. Salir");
    System.out.print("Seleccione una opción: ");
    opcion = entrada.nextInt();
    switch (opcion) {
        case 1:
            System.out.println("\n--- DATOS DEL JEAN ---");
            System.out.println("Código: " + codigo);
            System.out.println("Color: " + color);
            System.out.println("Talla: " + talla);
            System.out.println("Fue teñido: " + (fueTenido ? "Sí" : "No"));
            System.out.println("Cantidad de teñidos: " + cantidadTenidos);
            System.out.printf("Precio: $%.2f%n", precio);
            System.out.println("Cantidad de botones: " + cantidadBotones);
            System.out.printf("Humedad: %.2f%%\n", humedad);
            System.out.println("Estado de la tela: " + estadoTela);
            break;
        case 2:
            if (cantidadTenidos > 0) {
                cantidadTenidos--;
                if (cantidadTenidos == 0) fueTenido = false;
                System.out.println("Lavado realizado.");
            } else System.out.println("No quedan teñidos por disminuir.");
            System.out.println("Cantidad de teñidos: " + cantidadTenidos);
            break;
        case 3:
            if (humedad > 0) {
                humedad -= 10;
                if (humedad < 0) humedad = 0;
                System.out.println("Secado realizado.");
            } else System.out.println("El jean ya está seco.");
            System.out.printf("Humedad actual: %.2f%%\n", humedad);
            break;
        case 4:
            System.out.println("Programa finalizado.");
            break;
        default:
            System.out.println("Opción inválida.");
    }
} while (opcion != 4);
entrada.close();
}
}

```

C++

```
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;
int main() {
    string codigo, color, talla, estadoTela;
    int respuestaTenido, cantidadTenidos, cantidadBotones, opcionEstado, opcion;
    double precio, humedad;
    bool fueTenido;
    // Se registran y validan los datos del jean.
    cout << "Codigo del jean: "; getline(cin, codigo);
    while (codigo.empty()) {
        cout << "El codigo no puede estar vacio. Ingrese nuevamente: ";
        getline(cin, codigo);
    }
    cout << "Color: "; getline(cin, color);
    while (color.empty()) {
        cout << "El color no puede estar vacio. Ingrese nuevamente: ";
        getline(cin, color);
    }
    cout << "Talla: "; getline(cin, talla);
    while (talla.empty()) {
        cout << "La talla no puede estar vacia. Ingrese nuevamente: ";
        getline(cin, talla);
    }
    do {
        cout << "¿Fue tenido? (1 Si, 0 No): "; cin >> respuestaTenido;
        if (respuestaTenido != 0 && respuestaTenido != 1) cout << "Respuesta invalida.\n";
    } while (respuestaTenido != 0 && respuestaTenido != 1);
    fueTenido = respuestaTenido == 1;
    if (fueTenido) {
        do {
            cout << "Cantidad de tenidos (mayor que 0): "; cin >> cantidadTenidos;
            if (cantidadTenidos <= 0) cout << "Cantidad invalida.\n";
        } while (cantidadTenidos <= 0);
    } else cantidadTenidos = 0;
    do {
        cout << "Precio (mayor o igual que 0): $ "; cin >> precio;
        if (precio < 0) cout << "Precio invalido.\n";
    } while (precio < 0);
    do {
        cout << "Cantidad de botones (0 o mas): "; cin >> cantidadBotones;
        if (cantidadBotones < 0) cout << "Cantidad invalida.\n";
    } while (cantidadBotones < 0);
    do {
        cout << "Humedad del jean (0 a 100): "; cin >> humedad;
        if (humedad < 0 || humedad > 100) cout << "Humedad invalida.\n";
    } while (humedad < 0 || humedad > 100);
    do {
        cout << "Estado de la tela (1 Buena, 2 Regular, 3 Danada): "; cin >> opcionEstado;
        if (opcionEstado < 1 || opcionEstado > 3) cout << "Estado invalido.\n";
    } while (opcionEstado < 1 || opcionEstado > 3);
    if (opcionEstado == 1) estadoTela = "Buena";
    else if (opcionEstado == 2) estadoTela = "Regular";
    else estadoTela = "Danada";
    // Toda la lógica está en main, sin clases ni funciones adicionales.
    do {
        cout << "\n--- GESTION DEL JEAN ---\n";
        cout << "1. Mostrar datos\n2. Lavar jean\n3. Secar jean\n4. Salir";
        cout << "\nSeleccione una opcion: "; cin >> opcion;
        switch (opcion) {
            case 1:
                cout << "\n--- DATOS DEL JEAN ---\n";
                cout << "Codigo: " << codigo << "\nColor: " << color << "\nTalla: " << talla;
                cout << "\nFue tenido: " << (fueTenido ? "Si" : "No");
```

```

    cout << "\nCantidad de tenidos: " << cantidadTenidos;
    cout << fixed << setprecision(2) << "\nPrecio: $" << precio;
    cout << "\nCantidad de botones: " << cantidadBotones;
    cout << "\nHumedad: " << humedad << "%\nEstado de la tela: " << estadoTela << "\n';
    break;
case 2:
    if (cantidadTenidos > 0) {
        cantidadTenidos--;
        if (cantidadTenidos == 0) fueTenido = false;
        cout << "Lavado realizado.\n";
    } else cout << "No quedan tenidos por disminuir.\n";
    cout << "Cantidad de tenidos: " << cantidadTenidos << "\n';
    break;
case 3:
    if (humedad > 0) {
        humedad -= 10;
        if (humedad < 0) humedad = 0;
        cout << "Secado realizado.\n";
    } else cout << "El jean ya esta seco.\n";
    cout << fixed << setprecision(2) << "Humedad actual: " << humedad << "%\n";
    break;
case 4:
    cout << "Programa finalizado.\n";
    break;
default:
    cout << "Opcion invalida.\n";
}
} while (opcion != 4);
return 0;
}

```